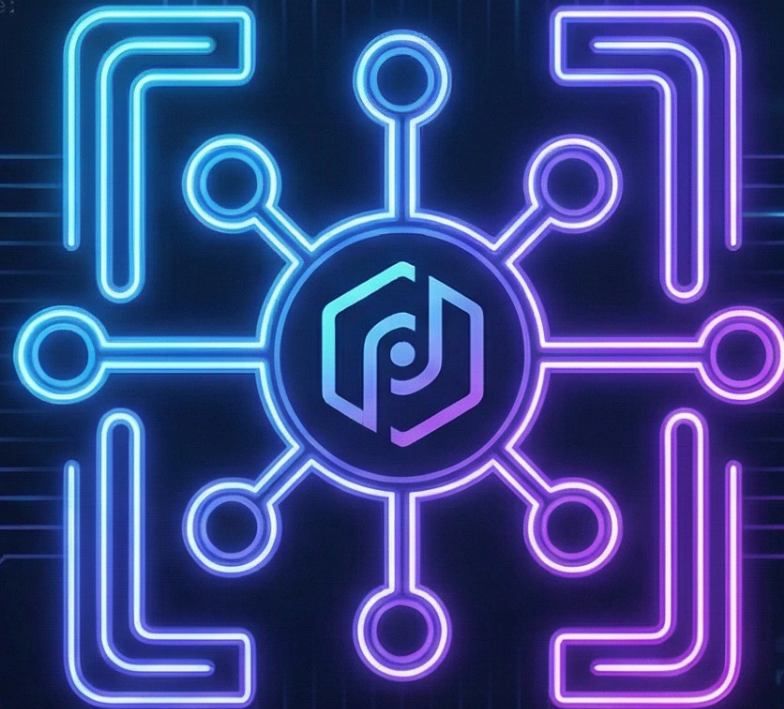


# PYDANTIC

Forzando a la IA a hablar  
en estructuras, no en prosa

Module: Desarrollo Avanzado de  
Sistemas Multiagente

Instructor: Rubén Juárez Cádiz



# ¿Qué aprenderemos hoy?



**1.** El problema del “texto libre” en los LLMs



**2.** ¿Qué es Pydantic y por qué es esencial?



**3.** Cómo Pydantic define contratos de datos



**4.** Integración con OpenAI Structured Outputs



**5.** Validación y manejo de errores automático



**5.** Validación y manejo de errores automático



**6.** Caso práctico: Extractor de entidades de soporte



**7.** El modelo TicketSoporte en acción



**8.** Entregable y criterios de evaluación

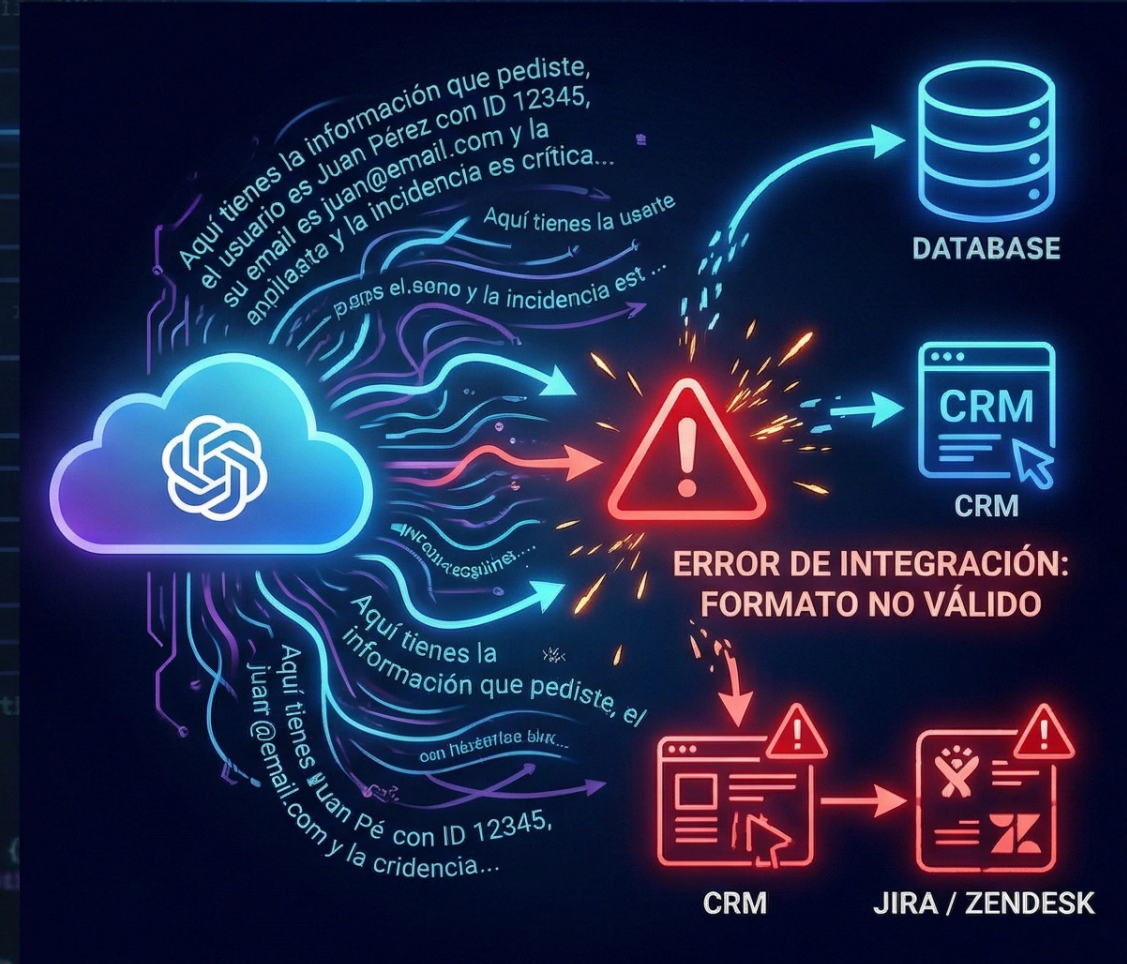


**9.** Próximos pasos y recursos

# Los LLMs devuelven **prosa**. Los sistemas de producción necesitan **datos**.

## Key Points

- El problema real: Un LLM responde en lenguaje natural. Perfecto para chat, desastre para automatización.
- ¿Cómo insertas texto libre en tu base de datos? No puedes. Rompe con cualquier cambio de formato.
- Las tres preguntas sin respuesta:
  1. ¿Qué campos exactos va a devolver la IA?
  2. ¿Qué tipo de dato tendrá cada campo?
  3. ¿Qué pasa si la IA decide omitir un campo?



# Pydantic define el contrato de <sup>JSON</sup> datos que la IA debe cumplir

## Key Points:

### ¿Qué es Pydantic?

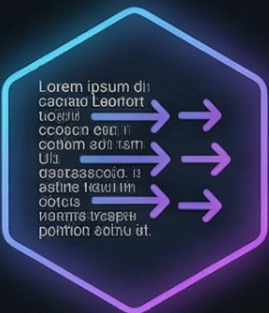
Librería de validación de datos más popular de Python (200M+ descargas/mes).

### ¿Cómo funciona?

Defines clases ("Modelos") que especifican campos y tipos.

### La magia:

Legible para humanos, ejecutable por Python, comprensible por LLMs.



Texto caótico



Pydantic Model



JSON estructurado perfecto

```
class TicketSoporte(BaseModel):  
    nombre_cliente: str  
    nivel_urgencia: Literal["Baja",  
                            "Media", "Alta", "Crítica"]  
    categoria_problema: str  
    requiere_reembolso: bool
```

# Pydantic garantiza que cada campo tenga el tipo correcto

|                                       |                                                            |                                                                            |
|---------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------|
| <code>str</code>                      | Nombre del cliente<br>(Texto de cualquier longitud)        | nombre: <code>str</code> = "Ana López" ✓                                   |
| <code>int</code> / <code>float</code> | Número de ticket, precio<br>(Solo valores numéricos)       | precio: <code>float</code> = 99.99 ✓ ticket_id: <code>int</code> = 12345 ✓ |
| <code>bool</code>                     | ¿Requiere reembolso?<br>(True/False, no "sí" o "no")       | reembolso: <code>bool</code> = <code>True</code> ✓ reembolso = "sí" ✗      |
| <code>list[str]</code>                | Lista de productos afectados<br>(Array de strings)         | productos: <code>list[str]</code> = ["item1", "item2"] ✓                   |
| <code>Literal[...]</code>             | Nivel de urgencia predefinido<br>(Solo valores permitidos) | nivel: <code>Literal</code> ["Baja", "Media"] ✓                            |
| <code>Optional[str]</code>            | Campo que puede ser nulo<br>(Permite None)                 | comentario: <code>Optional[str]</code> = <code>None</code> ✓               |

## El poder de `Literal`

```
nivel_urgencia: Literal["Baja", "Media", "Alta", "Crítica"]
```

La IA no puede devolver "urgente" o "5/10". Solo puede devolver exactamente uno de los cuatro valores definidos.

# Integración con OpenAI Structured Outputs

OpenAI lee tus modelos Pydantic y obliga al LLM a respetarlos

```
response = client.beta.chat.completions.parse(  
    model="gpt-4o-mini",  
    messages=[...],  
    response_format=TicketSoporte # ← La magia ocurre aquí  
)  
  
ticket = response.choices[0].message.parsed  
  
print(ticket.nombre_cliente)
```

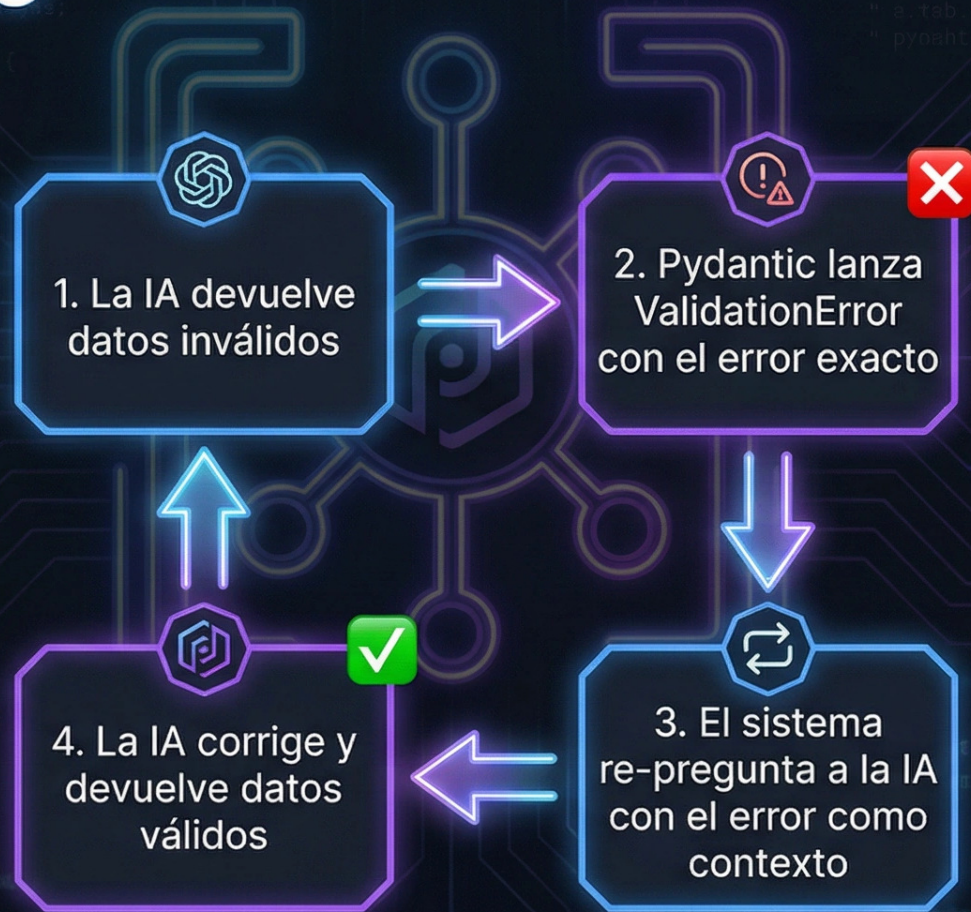


- El parámetro `response_format=TicketSoporte` convierte el modelo Pydantic en un JSON Schema.
- OpenAI usa este esquema para restringir la respuesta del LLM.

# Si la IA se equivoca, **Pydantic** detecta el error y permite corregirlo

Validación Automática y Recuperación de Errores

```
try:
    ticket = TicketSoporte(**datos_incorrectos) ❌
except ValidationError as e:
    print(e) ✅ Corrected: nivel_urgencia: 'Alta'
    # → 2 validation errors for TicketSoporte
    # → nivel_urgencia: Input should be 'Baja',
    #    'Media', 'Alta' or 'Crítica'
```



# Extractor Automático de Entidades de Soporte

## Caso Práctico: El Correo Caótico

**El Reto:** Transformar este correo real en datos estructurados para Jira/Zendesk

Lo que necesitamos extraer

[Correo Caótico]

De: pedro.martinez@empresa.com

Asunto: ESTO ES **INACEPTABLE!!!**

Hola, soy Pedro Martínez y llevo 3 días sin poder acceder a mi cuenta. Ya llamé dos veces y nadie me soluciona nada. El sistema me cobra doble en la factura de este mes. Necesito que me devuelvan el dinero **YA**. Esto es **urgente**, tengo una presentación mañana y sin acceso no puedo trabajar. Espero respuesta

**INMEDIATA.**

Pedro

|                       |                                 |
|-----------------------|---------------------------------|
| - nombre_cliente:     | "Pedro Martínez"                |
| - nivel_urgencia:     | "Crítica"                       |
| - categoria_problema: | "Acceso a cuenta / Facturación" |
| - requiere_reembolso: | True                            |

# El script completo: del correo caótico al JSON perfecto

## El Código del Extractor

```
1. import os
2. from dotenv import load_dotenv
3. from pydantic import BaseModel, Field
4. import openai
5.
6. # Importaciones y configuración (.env)
7. load_dotenv()
8. openai.api_key = os.getenv("OPENAI_API_KEY")
9.
10. # Definición del modelo Pydantic (TicketSoporte)
11. class TicketSoporte(BaseModel):
12.     cliente_id: str = Field(..., description="ID único del cliente")
13.     asunto: str = Field(..., description="Resumen del problema")
14.     prioridad: str = Field(..., description="Nivel de urgencia (Baja, Media, Alta)")
15.     descripcion: str = Field(..., description="Detalle completo del problema")
16.
17. # Definición del correo de entrada
18. correo_caotico = """
19. De: Juan Pérez <juan.perez@email.com>
20. Asunto: ¡Ayuda! No puedo acceder a mi cuenta
21. Hola soporte,
22. Mi ID de cliente es C12345. Desde ayer no logro iniciar sesión.
23. Es muy urgente, necesito acceder para trabajar.
24. Gracias, Juan.
25. """
26.
27. # Llamada a la API con response_format=TicketSoporte
28. response = openai.chat.completions.create(
29.     model="gpt-4-turbo",
30.     messages=[{"role": "user", "content": f"Extrae la información del siguiente correo:\n{correo_caotico}"}],
31.     response_format={"type": "json_object", "schema": TicketSoporte.model_json_schema()}
32. )
33.
34. # Impresión del resultado en formato JSON
35. print(response.choices[0].message.content)
36.
```

```
{
  "cliente_id": "C12345",
  "asunto": "No puedo acceder a mi cuenta",
  "prioridad": "Alta",
  "descripcion": "Desde ayer no logro iniciar sesión. Es muy urgente, necesito acceder para trabajar."
}
```

# El Resultado: JSON Perfecto para Cualquier API

De texto caótico a datos listos para producción en segundos

```
{  
  "nombre_cliente": "Pedro Martínez",  
  "nivel_urgencia": "Crítica",  
  "categoria_problema": "Acceso a  
cuenta y facturación duplicada",  
  "requiere_reembolso": true  
}
```

| Destinations                                                                                      |                                       |
|---------------------------------------------------------------------------------------------------|---------------------------------------|
|  Jira          | Crear issue con prioridad "Critical"  |
|  Zendesk       | Abrir ticket con urgencia y categoría |
|  Base de datos | INSERT directo sin parseo manual      |
|  Slack         | Alerta al equipo de soporte           |
|  CRM          | Actualizar ficha del cliente          |

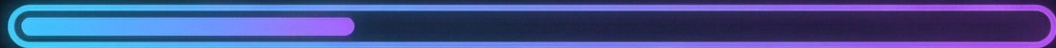
Un proceso que antes requería un agente humano leyendo correos ahora se ejecuta en **milisegundos**, con datos **perfectamente estructurados** y **sin errores**.

# Entregable y Criterios

Tu misión: Un extractor de entidades funcional con Pydantic

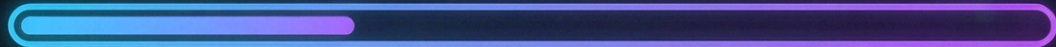
## Criterios de Evaluación

Modelo Pydantic (30%) 30%



Definición correcta con tipos y Literal

Integración OpenAI (30%) 30%



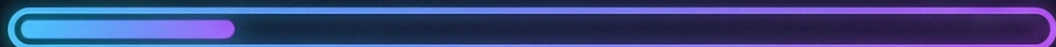
Uso de `response_format` con el modelo

Validación (20%) 20%



Manejo de `ValidationError` con `try/except`

Personalización (20%) 20%



Modelo extendido con al menos 2 campos extra

## Entregables Requeridos

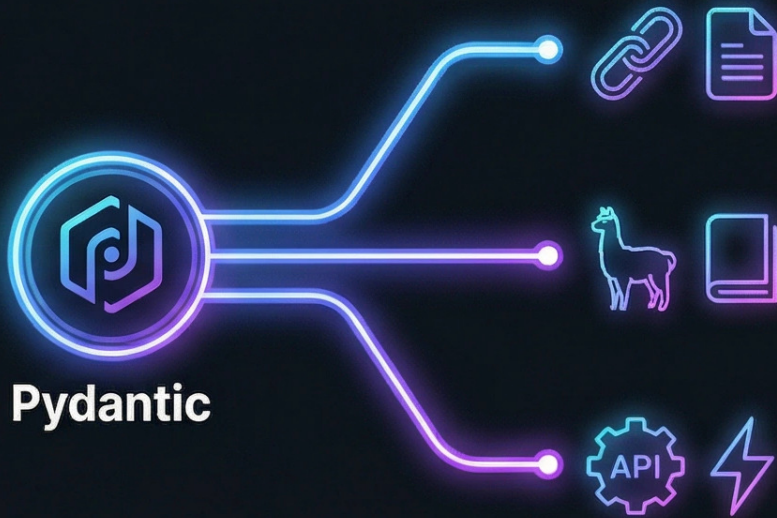
- ✓ 1. Archivo `extractor_soporte.py`
- ✓ 2. Al menos 3 correos de prueba **distintos (incluir uno en inglés)**
- ✓ 3. Salida JSON documentada para cada correo
- ✓ 4. Archivo `requirements.txt` y `.env.example`

## Extensión sugerida

Añadir campos como `idioma_correo`, `productos_afectados: list[str]`, o `numero_contacto: Optional[str]`.

# Próximos Pasos y Recursos

Pydantic es el puente. Los agentes estructurados son el destino.



## LangChain + Pydantic

Output Parsers para cadenas complejas

## LlamaIndex

Extracción estructurada de documentos

## FastAPI + Pydantic

Desplegar extractores como microservicios REST

## Recursos recomendados



Documentación oficial de Pydantic v2: [docs.pydantic.dev](https://docs.pydantic.dev)



OpenAI Structured Outputs: [platform.openai.com/docs/guides/structured-outputs](https://platform.openai.com/docs/guides/structured-outputs)



Repositorio del módulo: Disponible en el aula virtual

“

**Un agente que **no puede estructurar su salida** no puede integrarse con el mundo real.**

**Pydantic es el lenguaje común entre la IA y los sistemas de producción.**

— Rubén Juárez Cádiz

”